

iRallyMeter — what we changed

Saam & Claude · audit against SPEC-v2 · 4 August 2026 · for Amirali

Short version. Your architecture was right. The distance engine was already pure Dart, already single-integration speed-hold, already had automatic tunnel detection, and there was no Firebase to rip out. What we found were *specific* bugs inside a good design, plus five places where the code predated Revision 2 of the spec.

Nothing is merged. Everything is on two branches, SA-V1 and SA-V2 . Your main has not been touched once.

Where things stand

	Before	After
Tests passing	115	313
Tests failing	0	0
Analyzer warnings	2	2 (same two, untouched)
Test files	7	24
Release APK	never built	builds and runs

One test is still skipped on purpose, and its assertion has never been weakened. More on that under “What is still wrong”.

The bugs that actually mattered

Each one below is real, was reproduced, and has a number attached. They are in order of how much damage they did.

1. A faster GPS made the app measure *less*

The noise-gating rule threw away any movement smaller than the fix's own accuracy — but it also **moved the reference point** when it did that, so the movement was destroyed instead of ignored. The cutoff works out to $\text{accuracy} \times \text{fixRate}$, which means that at the 5 Hz the app itself asks for, **everything below 90 km/h was being discarded**.

It hid for fourteen steps because every test fixture ran at 1 Hz and highway speed. Fix: hold the reference point when the floor rejects a sample. Stop-and-go traffic went from **-5.04 %** to **-0.12 %**, and a parked car still accumulates exactly zero.

2. Estimation Mode could get stuck on, and invented distance while stuck

The app measures with any fix up to 25 m accuracy. But §15.2 only lets it *leave* Estimation Mode on a fix of 20 m or better, and nothing else ended the mode. So fixes between 21 m and 25 m were a trap: good enough

to measure with, not good enough to escape with. Light tree cover sits in exactly that band.

The damage is not noise. A dead-reckoning estimate holds the speed you entered at, so it cannot see the car slowing down. In the test: the car halved its speed and the app reported **2400 m for 1200 m of real travel**. That error is permanent, because the correction logic only ever adds distance back, never removes it.

Fix: a sustained run of usable, consistent fixes now also ends the mode. §15.2 is untouched — three good fixes still exit immediately.

3. A tunnel is silence, and the first fix made it worse

We found the location stream could never recover if you switched location services off and back on. The first attempt at a fix was a plain timeout, and it was **worse than the bug**: on the phone it restarted the Android foreground service about **twenty times inside one 400-second tunnel** — the exact service keeping the app alive in there.

Rebuilt so tearing the stream down needs *evidence*, not just quiet. Measured at **zero restarts** across 150 s and 90 s of silence.

4. iOS had never actually been configured

`_settings()` returned `AndroidSettings` on **every** platform. It compiled and ran because `AndroidSettings` is a `LocationSettings`, so the failure was completely silent. iOS was getting default accuracy, no activity type, and `pauseLocationUpdatesAutomatically` left at its default — which pauses location whenever iOS decides the car has stopped. That is every start line and every time control.

5. Resetting a trip leaked the previous leg into the new one

Up to **715 m** of the old leg's tunnel correction was still being paid out into the new leg. On a rally that is the difference between “turn after 3 km” landing on the right junction and the wrong one.

6. The compass — three separate causes

You said it was laggy and inaccurate. It was all three of these at once:

- the smoothing used a per-sample weight, so the lag was a property of the handset's sensor rate rather than of time
- **“Use true north” only changed a label.** No declination was ever applied, so it showed a magnetic reading and called it true
- tilt compensation worked out “which way is down” from the raw accelerometer, so it followed braking and cornering — wrong exactly when you are looking at it

That same per-sample-filter mistake turned up three times in the codebase: speed display, compass needle, compass tilt. Worth watching for.

7. The app could fail to start at all

`main()` waited on two permission requests before `runApp()` with no error handling. On a first install they raced, threw, and the app hung on the Flutter logo forever.

What else changed

- **Manual TUNNEL buttons deleted** (SPEC-v2 §14/§15) — detection is fully automatic now, with a section log you can read in Settings.
- **Speed comes from GNSS Doppler**, with position differentiation only as a fallback when Doppler is untrustworthy.
- **Estimated state is visible** — amber digits plus an `EST` / `EST?` / `SYNC` badge, on all four readouts. The badge is *text*, not just colour, because at night the whole palette is one hue family and colour alone tells you nothing.
- **Portrait no longer overflows**. Your own skipped test recorded 142 px, and it grew with the status text — 137 px in Estimation Mode. The nav icons now sit on their own row in portrait only. Nothing was shrunk, and the touch targets ended up further apart.
- **Permissions are explained before they are asked for**, which both stores require and which was a rejection risk.
- **Removed `ACCESS_BACKGROUND_LOCATION`** — it was declared in the manifest and never once requested, so it was never granted and did nothing except add store-review burden.
- **A record-and-replay harness** with nine synthetic drives, so accuracy claims are measured against known ground truth instead of asserted.

What is still wrong — please read this part

These three are not fixed, and none of them can be fixed from a desk.

1. **Urban canyon under-reads by 12.50 %** against the spec's 1 % target. Its test is skipped but written at *full* strength — deliberately never weakened. We did not tune this, because tuning against a synthetic fixture with no real-world number to check against is guessing, and the same threshold is what keeps a parked car at exactly 0.000 km.
2. **No road test has ever happened**. Every accuracy figure in this document comes from generated fixtures where the true answer is exact by construction, or from an emulator. A better replay is not a drive.
3. **The app has never run on iOS**. It compiles for the simulator. That is the entire extent of it.

What we need from you

Ten decisions. Full detail is in `docs/QUESTIONS-FOR-AMIRALI.md` ; this is the summary.

Blocking — work stops until these are answered

	Question
B2	Which map tile source? This blocks offline maps completely, and offline is a hard requirement for rally use. The app currently pulls tiles live from OpenStreetMap, and the OSMF policy explicitly prohibits the pre-seeding your existing <code>FileTileProvider</code> TODO plans — so that TODO cannot be finished as written.

	Options are ranked in <code>docs/IRAN-CONSTRAINTS.md</code> §4; our guess is Neshan or Map.ir with a side-loaded pack.
B8	Whose signing key, and where is it kept? The release APK is currently signed with the debug keystore. Fine for handing to your dad. Fatal for Cafe Bazaar or Myket — and an app first installed under a debug key cannot be updated with a real one without uninstalling , which wipes every tester's trips. This gets worse the longer you leave it.
B3	Do Iranian crews want Persian-Indic digits, or Latin numbers with translated labels? This is the one we genuinely cannot answer. Rally road books and tripmeters are conventionally read in Latin digits, and the digits <i>are</i> the product. There is no localisation at all right now.

Confirmations — already built this way, say if you disagree

	Question
A1	§12.2's $\pm 25\%$ clamp saturates at the ceiling rather than snapping back to v_0 . We built the literal reading first and our own test caught that it sawtooths — 25, 20, 25, 20 — which on a driver-facing readout looks like a fault.
A2	The $\pm 25\%$ bound is applied upwards only , so braking runs all the way to a stop. Symmetrical, a car braking to a halt in a tunnel would hold v_0 and invent distance — and your own <code>tunnel_system_test</code> 11 already asserts a clean stop.
B1	Portrait is supported and the nav icons now take a second row. Happy with that, or would you rather portrait dropped something instead?
B6	Confirm <code>minSdk = 23</code> and <code>IPHONEOS_DEPLOYMENT_TARGET = 12.0</code> are deliberate floors. Flutter's migrators keep raising both on every native build and we keep reverting them.

Product calls — not blocking

	Question
B4	§8 names both a moving average and an overall one. We built overall-only and labelled the tile <code>AVG (ALL)</code> . Add the second?
B5	Is iOS a shipping platform, or Android-first? Iranian third-party iOS stores re-sign your binary and Apple revokes those certificates, which kills every installed copy at once.
B7	A denied permission currently has no in-app way back — the screen points at Android Settings. Add a “grant permission” button in Settings? We lean yes: someone who taps DON'T ALLOW by accident has a dead app and no clue why.

The road test — the part only a car can do

Full protocol in `docs/ROAD-TEST.md`. Four things, in priority order. Open **Settings** → **Section log** for the GNSS HEALTH panel and screenshot it.

1. **Mid-drive, switch location services off for a minute, then back on. Do not restart the app.** The trip counter must start advancing again. This is the one fix whose code path is certain but whose on-device behaviour we could not reproduce either way on an emulator. **If this fails, Phase 3 reopens** — it is not a small ticket.

2. **A real tunnel** (Niayesh or Alborz). Distance within about 3 %, no jump or reversal on exit, and `STREAM STALLS` must still read 0.
3. **50 km against measured markers**. The target is 1 %.
4. **City traffic and a twisty road**. These do not have to pass. They have to be *measured*, because those two numbers are what decide whether we touch the noise-gating threshold at all.

Numbers, not impressions. If something fails, send the number and a screenshot of the section log. The two that are stop-ship are a trip counter that freezes and does not recover, and any visible jump or reversal when you come out of a tunnel.

Where to look

<code>docs/AUDIT-REPORT.md</code>	The full technical record
<code>docs/QUESTIONS-FOR-AMIRALI.md</code>	The ten questions, in detail
<code>docs/ROAD-TEST.md</code>	The drive protocol
<code>docs/TEST-BUILD.md</code>	How to build and hand over a test APK
<code>docs/IRAN-CONSTRAINTS.md</code>	Distribution, tiles, Play Services, localisation
<code>docs/SECURITY-REVIEW.md</code>	Permissions and location-data handling
<code>AGENTS.md</code>	Conventions, and the traps that cost us time

One more review pass is still to come: an independent adversarial review of the estimation state machine and the background location lifecycle. Anything it finds gets fixed before the pull request is opened. Nothing will be merged without you.